

PocketTrack — Claude Code Audit & Launch Package

Complete Gmail Sharing Protocol, "Zero-Friction" Claude Run Setup, and Multi-Persona Audit Guide

Vite + React 19

Dexie (IndexedDB) + Supabase

Clean Zip: 154 KB

Instant "Just Hit Run" Setup

Antigravity Handoff Ready

⚡ Instant "Unzip & Hit Run" Instructions for Your Friend

We embedded both **CLAUDE.md** and **AUDIT_PROMPT.md** directly inside the root of **PocketTrack_Project.zip**! Your friend doesn't need to manually configure anything:

1. **Unzip:** Extract **PocketTrack_Project.zip** into any folder.
2. **Open Terminal in that folder and run:**
`npm install` (takes ~10 seconds to install dependencies)
`claude` or `claude "Run the full audit from AUDIT_PROMPT.md"`
3. **Hit Enter:** Claude Code automatically detects **CLAUDE.md**, reads all source files, and produces the complete Multi-Persona Audit & Antigravity Cards!
4. **Send Back:** Your friend copies the generated **[ANTIGRAVITY TASK]** cards and emails or messages them to you!

1. Sending the Files via Gmail (Without Exceeding Limits)

File Ready on Desktop: C:\Users\kavit\OneDrive\Desktop\PocketTrack_Project.zip

File Size: Only 154 KB — attaches in less than 1 second to Gmail (limit is 25 MB)!

- **Why standard zips fail:** Zipping an entire project folder includes `node_modules/` (over 50,000 files, 300MB–800MB), causing Gmail to reject it immediately.
- **How this zip is built:** Excludes `node_modules/` and `dist/`, but retains 100% of the source code, configs, **CLAUDE.md**, and **AUDIT_PROMPT.md**. Your friend's machine automatically installs the packages using `npm install`.

Ready-to-Send Note to Copy-Paste into Gmail:

"Hey! Attached is our app codebase (**PocketTrack_Project.zip**, 154KB) plus the full PDF Audit Guide.

To review it in Claude Code:

1. Unzip the archive and open terminal in that folder.
2. Run `npm install`.
3. Run `claude` or `claude 'Run the full audit from AUDIT_PROMPT.md'`.
4. The prompt and 7 personas are pre-loaded in **CLAUDE.md** and **AUDIT_PROMPT.md**! Send me the Antigravity task cards when done!"

2. The 7 Review Personas Configured for Claude

Claude will evaluate your app through 7 specialized human roles simultaneously:

**1. Principal QA & Code Auditor**

Inspects React 19 hooks, Dexie local DB vs Supabase cloud sync race conditions, offline handling, mobile viewport keyboard overlap, and floating-point math rounding errors.

**2. College Student / Gen-Z Youngster**

Demands 3-second rapid expense logging, impulse purchase cooling periods, peer bill-splitting, celebratory audio/confetti feedback, and zero clunky financial jargon.

**3. Middle-Class Family Teacher**

Evaluates 50/30/20 envelope budgeting, grocery and utility inflation alerts, emergency savings safety nets, and stress-free visual clarity that reduces anxiety.

**4. Certified Financial Planner (CFP)**

Verifies mathematical compounding accuracy in the Wealth Simulator, FIRE retirement runway calculations, inflation adjustments, and net worth balance sheet integrity.

**5. World-Class UI/UX Creative Director**

Audits dark-mode luxury aesthetics, glassmorphism, contrast accessibility (WCAG AA), typography, micro-interactions, and thumb-friendly mobile reachability.

**6. "Healthy Addiction" Psychologist**

Applies Nir Eyal's Hook Model (Trigger → Action → Variable Reward → Investment) with streak freezes, milestone badges, and daily financial mindfulness loops.

**7. Chief Legal & Launch Operations Director**

Ensures complete launch readiness: Privacy Policy for financial data, Terms of Service, GDPR/CCPA data export/deletion, non-advisory regulatory disclaimers, and Supabase Row Level Security (RLS) audits.

3. Master Audit Prompt (Embedded in AUDIT_PROMPT.md & CLAUDE.md)

If your friend uses the Claude Web UI instead of the CLI, they can simply copy and paste this entire block:

```
=== COPY-PASTE PROMPT FOR CLAUDE (CLAUDE CODE / FABLE 5.1 / SONNET) ===

You are an elite Multi-Persona Product, QA, and Engineering Review Board auditing the "PocketTrack" personal finance and wealth management application.

Inspect all source files in `src/` (components, hooks, db, context, lib) and adopt the following 7 DISTINCT HUMAN PERSONAS simultaneously. Deliver a deep, rigorous, and comprehensive review covering bugs, launch readiness, viral gamification, and world-class UI design:

1. 🔍 PRINCIPAL QA ENGINEER: Audit React 19 + TypeScript + Dexie + Supabase. Detect race conditions, offline desync, memory leaks, mobile viewport bugs, and floating-point math rounding errors. Deliver a categorized Bug Report Table (Severity, File, Root Cause, Exact Fix).
2. ⚡ GEN-Z / STUDENT: Evaluate 3-second quick logging, impulse spending cool-down timer, peer bill splitting, audio/confetti celebratory feedback, and streak gamification.
3. 🎓 FAMILY TEACHER: Evaluate 50/30/20 envelope budgeting, grocery inflation alerts, emergency fund progress, and stress-free, jargon-free visual clarity.
4. 💼 CFP WEALTH ADVISOR: Verify compounding math in the Wealth Simulator, FIRE retirement runway, inflation models, and asset/liability net worth accuracy.
5. 🎨 UI/UX CREATIVE DIRECTOR: Propose 5 breathtaking dark-mode visual and micro-interaction upgrades (glassmorphism, typography, tactile rhythm).
6. 🚀 BEHAVIORAL PSYCHOLOGIST: Design ethical, "good-addictive" habit loops (Nir Eyal's Hook model, streak freezes, financial mindfulness cards).
7. ⚖️ LEGAL & LAUNCH DIRECTOR: Provide mandatory launch requirements (Privacy Policy, Terms of Service, GDPR/CCPA 1-click export/wipe, financial disclaimer, Supabase RLS security).
```

```
=====
CRITICAL OUTPUT REQUIREMENT: ANTIGRAVITY PROMPT CARDS
=====
```

At the end of your report, you MUST convert all your recommendations into copy-pasteable "ANTIGRAVITY PROMPT CARDS". The developer will paste these cards straight into Google DeepMind Antigravity to build or fix each item automatically:

Format each card exactly like this:
```

```
[ANTIGRAVITY TASK:]
Files to Modify:
Task Description:
Expected Behavior:
```
```

Now, inspect the codebase and generate the complete report!

=== END OF PROMPT ===

4. Production Launch Readiness & Legal Checklist

Domain	Requirements & Deliverables	Launch Impact
1. Legal Policies	• Privacy Policy: Disclosing Dexie IndexedDB storage, Supabase authentication, and telemetry. • Terms of Service: Limiting liability and establishing user conduct. • Regulatory Disclaimer: "PocketTrack is an educational budgeting tool and does not provide certified financial, tax, or investment advice."	Mandatory for App Store, Google Play, and payment gateway approval.
2. Compliance (GDPR/DPDP)	• 1-Click Data Export: Download all records in JSON and CSV format. • Account Wipeout: Irreversible deletion of cloud data and local IndexedDB database.	Legally required by global data privacy acts.
3. Security & Database	• Supabase Row Level Security (RLS): Verify user isolation policies. • Key Protection: Ensure only anon public keys are used in client bundles.	Prevents unauthorized data access and breaches.
4. Packaging & PWA	• Web App Manifest: Icons (192x192, 512x512) and offline caching. • Capacitor / Tauri Wrapper: For native iOS and Android store distribution.	Enables home-screen installation and native device feel.

5. How to Bring Claude's Output Back to Antigravity

Zero-Coding Workflow:

Whenever your friend sends you Claude's report, look for the **[ANTIGRAVITY TASK]** cards. You do not need to write any code! Simply copy the text of each card and paste it directly into our Antigravity chat.

Example Handoff:

You paste this into Antigravity:

Hey Antigravity, Claude Code audited our app and recommended this task. Please implement it:

[ANTIGRAVITY TASK: Impulse Purchase Cool-Down Timer]

Files to Modify: src/components/transactions/TransactionModal.tsx, src/db/index.ts

Task Description: Add a "Cool-down" toggle for discretionary expenses over \$50. If checked, save transaction as 'pending_cool_down' with a 48-hour countdown badge on the dashboard.

Expected Behavior: If user confirms cancel after 48 hours, trigger celebratory soundFx and offer to transfer amount into Savings Goal.

What Antigravity Will Do Automatically:

1. Inspect the target files and dependencies.
2. Implement clean TypeScript components and state hooks.
3. Preserve existing features while applying modern styling.

4. Validate the build with `npm run build` to ensure zero TypeScript errors.